Laboratory 1: basic peripherals on a ST microcontroller
System-On-Chip Architecture

Andrea Pignata
andrea_pignata@polito.it

Walter Gallego Gomez
walter.gallego@polito.it

Gianvito Urgese
gianvito.urgese@polito.it

November 14, 2025

Abstract

In this laboratory session, we will explore the basic usage of some peripherals in the STM32 Nucleo-F401RE board, including GPIOs, NVIC, Timers, DMA and UART. We will use CubeMX for the initial configuration, the stm32pio tool to convert the project to a platformio compatible one, and Renode for simulation. Students will also have the opportunity to test the results on real hardware.

Keywords: *Peripherals, GPIOs, NVIC, Timers, DMA, UART, CubeMX, Platformio, Renode*

Contents

1 Materials **2**

2 Guided section **2**
2.1 Environment configuration . . . 2
2.1.1 Required tools . . . 2
2.1.2 Installing and configuring the tools . . . 3
2.2 GPIOs and interrupts . . . 3
2.2.1 Creating a project in CubeMX . . . 3
2.2.2 Configuring GPIO pins . . . 5
2.2.3 Configuring the Interrupts . . . 6
2.2.4 Configuring and Saving the project . . . 8
2.2.5 Code generation with stm32pio . . . 9
2.2.6 Personalizing your Firmware . . . 10
2.2.7 Compiling . . . 12
2.2.8 Button and LED in Renode . . . 12
2.2.9 (Optional) Testing with the real hardware . . . 13

3 Homework **14**
3.1 Timers . . . 14
3.1.1 Fixed blinking frequency . . . 14
3.1.2 Changing the blinking frequency at runtime . . . 14
3.2 DMA and UART . . . 15
3.2.1 DMA self-study . . . 15
3.2.2 Adding the UART . . . 16
3.3 Interrupt priorities . . . 17

4 Troubleshooting **18**
4.1 stm32pio issues . . . 18
4.2 PlatformIO issues . . . 18
4.3 Renode simulation issues . . . 18

1 Materials

This guide is accompanied by Renode platform description and script files for the STM32 Nucleo-F401RE board.

2 Guided section

2.1 Environment configuration

2.1.1 Required tools

- Linux OS is optional but recommended. In any case, students from previous years have successfully completed the labs with Windows and macOS.
- Visual Studio Code: https://code.visualstudio.com/ with PlatformIO plugin.
- STM32Cube MX: https://www.st.com/en/development-tools/stm32cubemx.html
- Renode: https://renode.io/

  Windows users: Be sure to check the Add to PATH option during installation.
- stm32pio: https://github.com/ussserrr/stm32pio

We will simulate the STM32 Nucleo-F401RE board using Renode.
https://www.st.com/en/evaluation-tools/nucleo-f401re.html
At the end of the laboratories, you will be able to deploy your final firmware on a real board and compare it to the simulated one. Figure 1 summarizes the development flow we will follow.

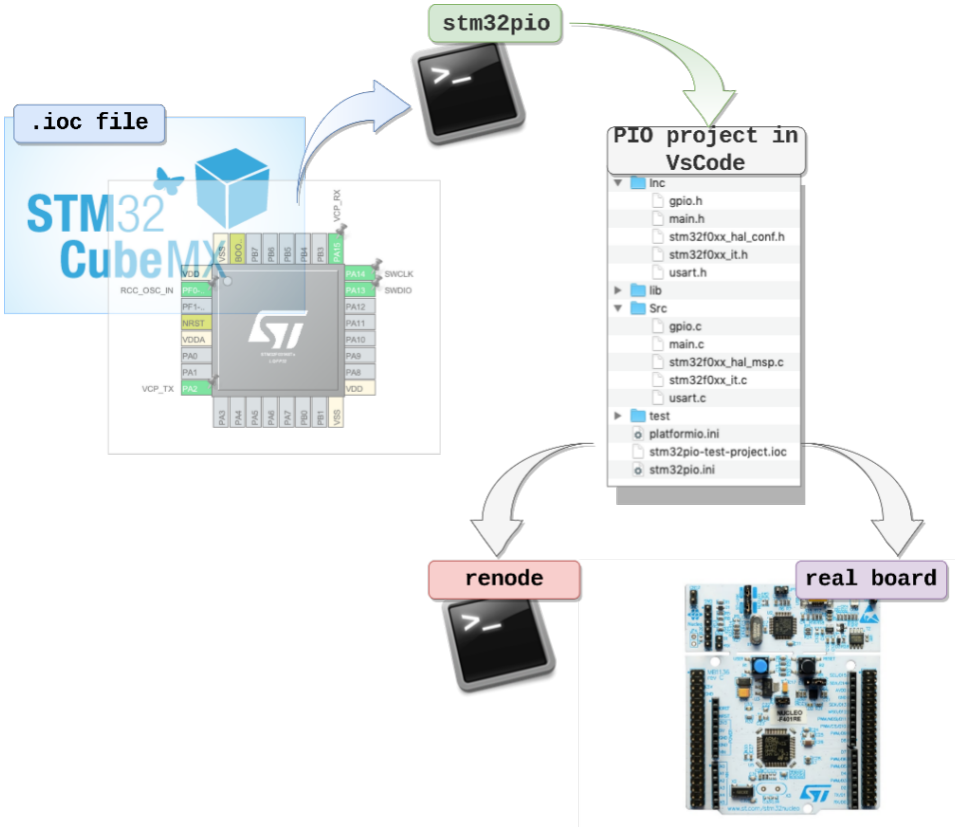


Figure 1: Development flow

2.1.2 Installing and configuring the tools

1. Install VsCode and the PlatformIO plugin (open VsCode → extensions icon on the left → search and install PlatformIO)
2. Install CubeMX

   Windows users: Install the CubeMX tool for **all users** and leave the default destination folder.

   Linux users: Leave the default destination folder.

   If you already have CubeMX installed, in a different PATH as the default, you will have to configure smt32pio later (see section 2.2.5).
3. Install Renode
4. Copy the Renode platform and script files we provided in a folder. That folder will become your workspace, where you will add all your firmware files.
5. (Optional) create a python environment with the tool of your choice, like conda, and a python version that supports stm32pio, like python=3.10.15. Enable the created environment.
6. Install stm32pio with:

```
$ pip install stm32pio
```

7. Configure your PATH so that the PlatformIO CLI installed with VsCode can be accessed from the terminal. Refer to https://docs.platformio.org/en/latest/core/installation/shell-commands.html

   For Linux, it is enough to link the executables installed by the PlatformIO VsCode plugin into your `$HOME/.local/bin/ folder`

```
ln -s ~/.platformio/penv/bin/platformio ~/.local/bin/platformio
ln -s ~/.platformio/penv/bin/pio ~/.local/bin/pio
ln -s ~/.platformio/penv/bin/piodebuggdb ~/.local/bin/piodebuggdb
```

   For Windows, you need to edit the system environment variable called Path and append the Platformio Scripts path (Usually `C:\Users\UserName\.platformio\penv\Scripts\`) in the beginning of the list (please replace UserName with your account name).

   Go to **Control Panel → Advanced System Settings → Environment Variables**

   In System variables, select the Path variable, click Edit and add the path specified above, as in Figure 2

   NOTE: For both Linux and Windows, when you update the PATH variable, programs already opened (Like VSCode or the Terminal) do not get the update. Reopen them so the new value of the PATH variable is loaded.

2.2 GPIOs and interrupts

You will learn to:

- Configure a basic project using ST Microelectronics tools.
- Manage interrupts.
- Start using the Hardware Abstraction Layer APIs by ST.

2.2.1 Creating a project in CubeMX

The suggested way to start working on a STM32 board is to use the CubeMX tool. This tool can **automatically generate initialization** code for the microcontroller and all the peripherals you want to use in your project, based on your preferences. In this way:

- You do not need to know everything about the peripherals available on the chip, the tool will list them and their characteristics to you.

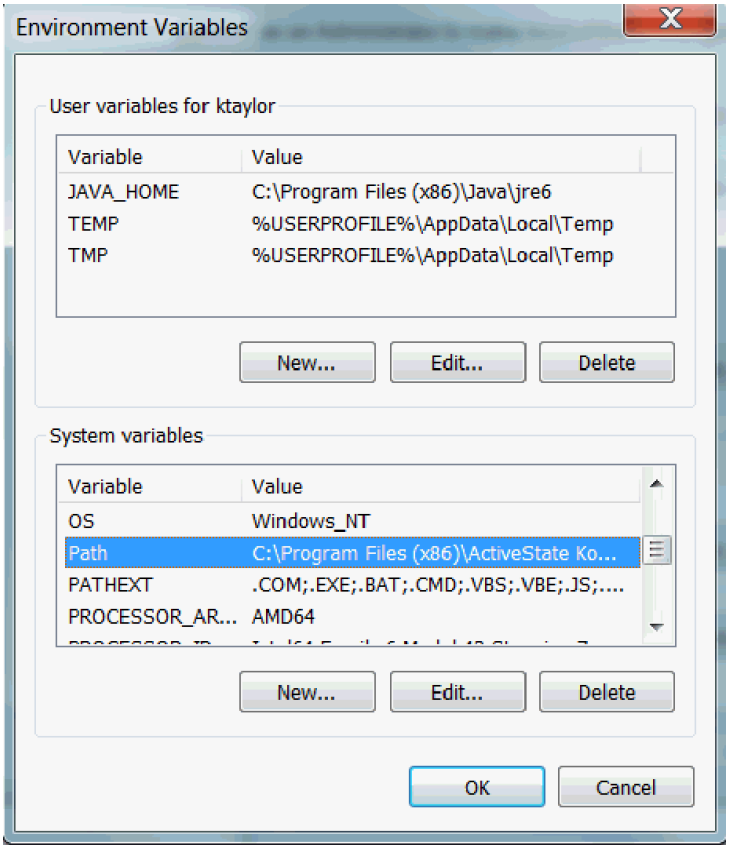


Figure 2: Windows Path Configuration.

- If you have something connected to a GPIO Pin, CubeMX will create a `#define` for the pin number, so you do not need to remember it.
- The initialization code is by default secure and working (...if you configured CubeMX correctly).
- You can jumpstart working on the functionality of the firmware, saving time.

Let's start creating the project on CubeMX.

1. Launch STM32CubeMX.
2. Under New Project, you will see three possibilities:
   (a) Access to MCU Selector: you can choose the Microcontroller Unit you are using. This is useful if you are developing your own PCB with an ST Microcontroller: CubeMX will ignore all the components you may want to connect to the microcontroller, and those will be up to you.
   (b) Access to Board Selector: you choose this if you want to start working with a development board produced by ST. In this way CubeMX will know how the MCU is connected to the other components and will initiate them for you.
   (c) Access to Example Selector: this option is useful if you want to start from a Firmware Example developed by ST and personalize it to fit your needs.

   In our case, we want to access to board selector.
3. In the Commercial Part Number field, put NUCLEO-F401RE. You will find the board we will work with on the bottom right part of the window: double click on it and then Start Project. Click on Yes when the tool asks *Initialize all peripherals with their default mode?*

4. The STM32F401RETx chip will warmly welcome you. You will see all its pins, some of them labeled with the functionality they have been assigned to. For example, you can see on the bottom that pin PA5 is assigned to the LED of the board (Fig 3).

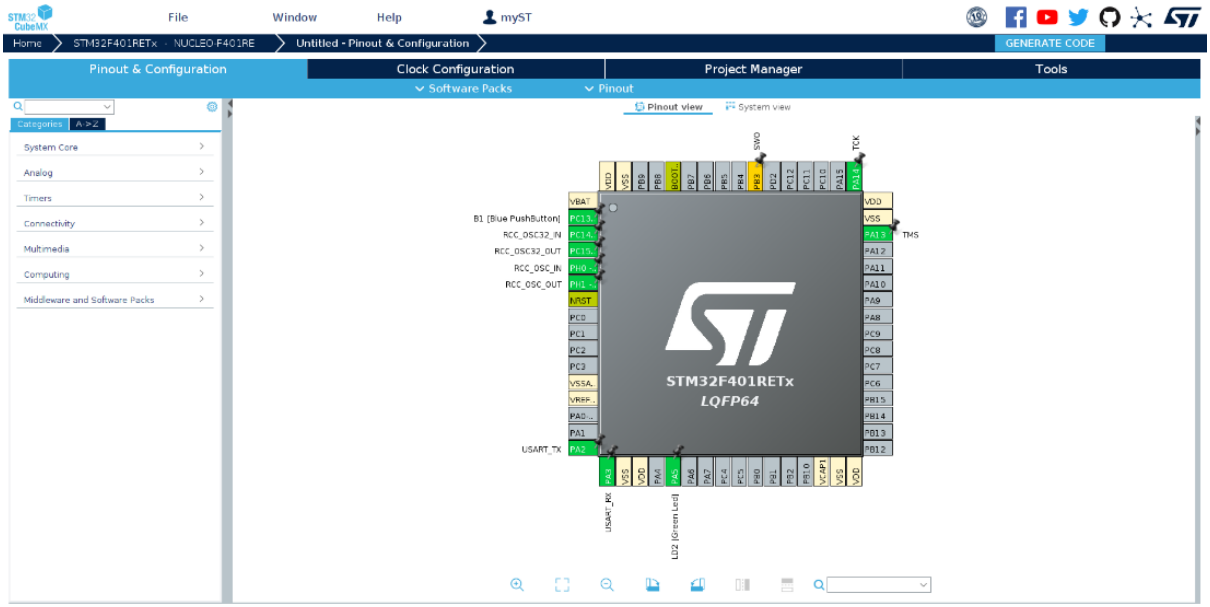


Figure 3: STM32F401RETx layout in CubeMX

2.2.2 Configuring GPIO pins

GPIO pins (General Purpose Input Output) are pins of the microcontroller that can be used as input or outputs by the firmware.

When used as outputs, the firmware is able to set the pins HIGH (a voltage of 3.3V will be supplied to the PIN) or LOW (the pin will be short-circuited to ground). When used as inputs, the firmware can read if the pin is HIGH (a voltage over a certain threshold is supplied) or LOW (the supplied voltage is lower).

You **must** pay attention to the **current** flowing through the pin and to the voltages applied to it. The chip datasheet reports the maximum ratings of current/voltage that a pin can support before damaging the chip. Always read it. An example: always connect an LED with a resistor in series, to limit the current according to Ohm's Law.

Now, let's dive into CubeMX again.

1. Search GPIO in the peripherals on the left of the window (in the Pinout and Configuration tab).
2. Click, for example, on PA5. You can see from the User Label that it is a LED: CubeMX already configured it as OUTPUT, with an initial output of LOW, and low speed (Fig 4).
3. What if you want to configure another PIN as GPIO, for example to add an external LED? You will need to find that PIN on the chip drawing on the right, click on it, and choose an option between `GPIO_Input`, `GPIO_Output` and `GPIO_Analog`. Then, it will appear on this list.

   We will not need to do so for this laboratory lesson.
4. Let's configure the button. Press on PC13, as you can see it is labeled as B1 (Blue Pushbutton).

   We want the MCU to **generate an interrupt** whenever the button is pressed.

   So, choose *External Interrupt Mode with Rising Edge Trigger Detection* as the *GPIO Mode*.

Questions for you:

- If you want to turn on an LED connected to a GPIO, should I put it LOW or HIGH?
- Is it the same if I choose *Falling edge detection* as a trigger of the interrupt for the button?
- What if I choose *Rising/Falling edge detection*?

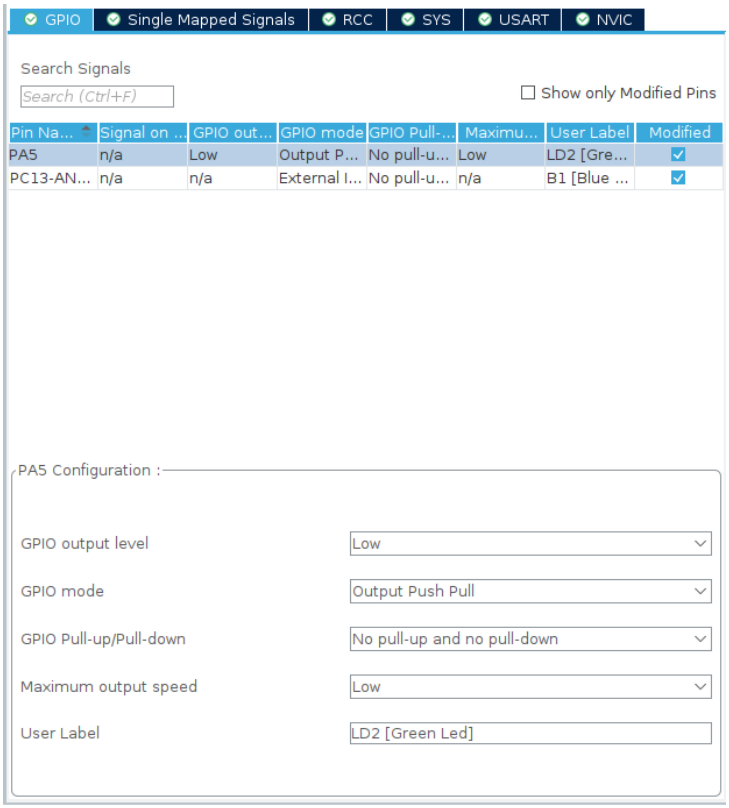


Figure 4: GPIO A5 configuration

2.2.3 Configuring the Interrupts

Microcontrollers use a particular **peripheral** to manage interrupts. This because:

- Interrupts should be **managed externally** from the microcontroller code, that should remain focused on the task it's performing.
- Interrupts have different **priorities**, and the corresponding queue should be managed as fast as possible in case there are multiple interrupts to be served.
- The MCU core could **disable all interrupts** in some cases, but the interrupt sources could be still active in that moment.

In ST Microelectronics products, that peripheral is called **NVIC** *Nested vector interrupt control*. How to configure it in CubeMX?

1. Search for the NVIC. A table like in Fig 5 will show up. You can see that all interrupts have the same priority.
2. Enable the *EXTI line[15:10]* Interrupt. That one is the interrupt generated by the GPIO pins.
3. Go to the Code Generation tab. As you can see in Fig 6, CubeMX is configured to create the code for the EXTI interrupt, for the following steps:
   - Initialize the Interrupt.
   - Generate the IRQ Handler.
   - The call to the HAL Handler.

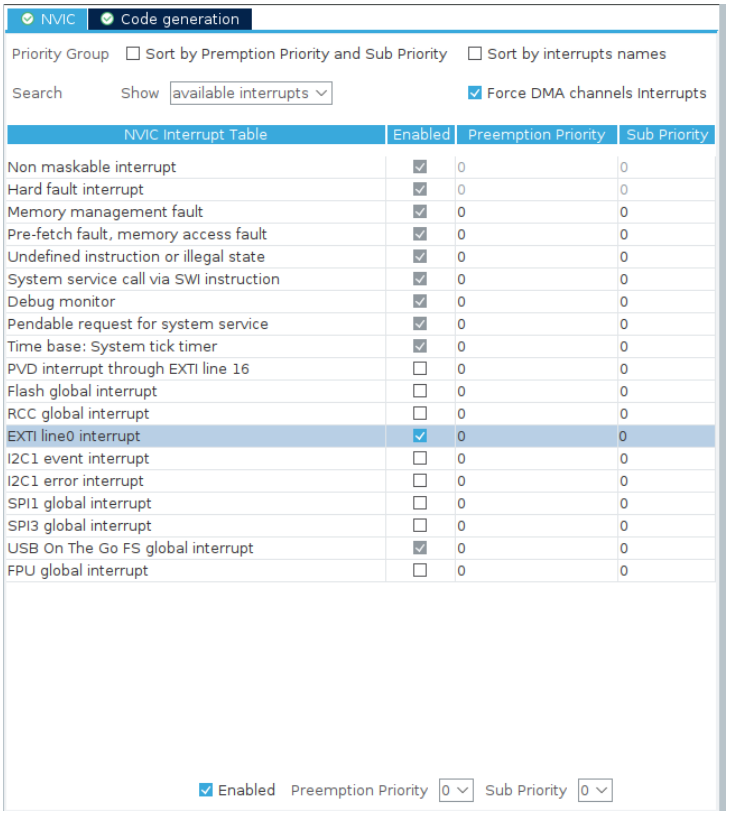


Figure 5: NVIC table in CubeMX

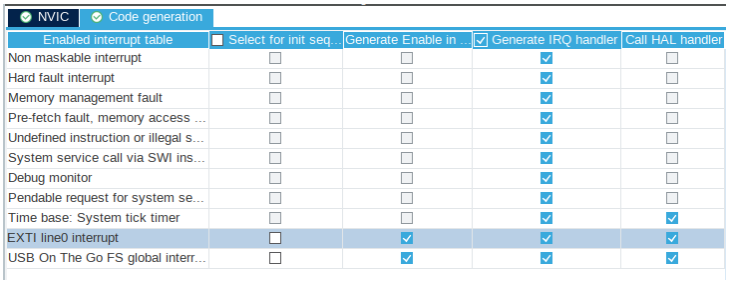


Figure 6: NVIC code generation CubeMX

The workflow in Figure 7 summarizes the basic operation of the EXTI interrupt (from https://wiki.st.com/stm32mcu/wiki/Getting_started_with_EXTI)

1. At init time, the NVIC is configured to listen to possible interrupts on the EXTI LINE.
2. We configured the GPIO to generate an interrupt in rising edge
3. Create a rising edge by pressing the button.
4. When an interrupt is generated, the NVIC will call the IRQ (Interrupt Request Handler), that has maximum priority and will disable all interrupts for all of its execution.
5. The IRQ will then call a lower-priority and not-exclusive HAL Handler.
6. The HAL handler will finally invoke a callback, that you can personalize in your firmware, to perform some custom action.

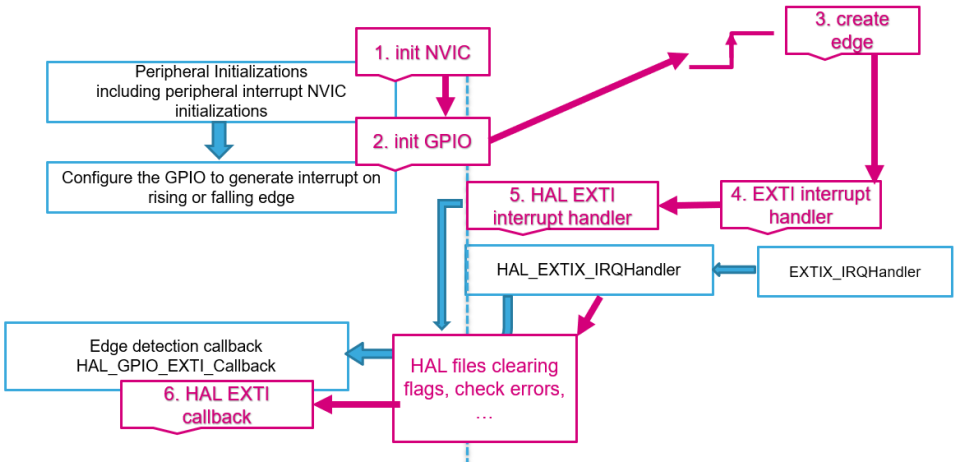


Figure 7: EXTI operational flow

2.2.4 Configuring and Saving the project

1. Click on **Project Manager** → **Project**, give it a name. Choose your workspace in Project Location.
2. Choose **STM32CubeIDE** as IDE. (This will work for PlatformIO as well) (Fig 8).
3. In the **Code Generator** tab check *Copy only the necessary library files* and *Generate periphery initialization as a pair of .c/ .h files per peripheral* options (Fig 9).
4. Save the project in your workspace with **File** → **Save Project**.

   NOTE: Make sure the path where you save your project does not contain any spaces. Paths with spaces may generate issues when running the simulation.

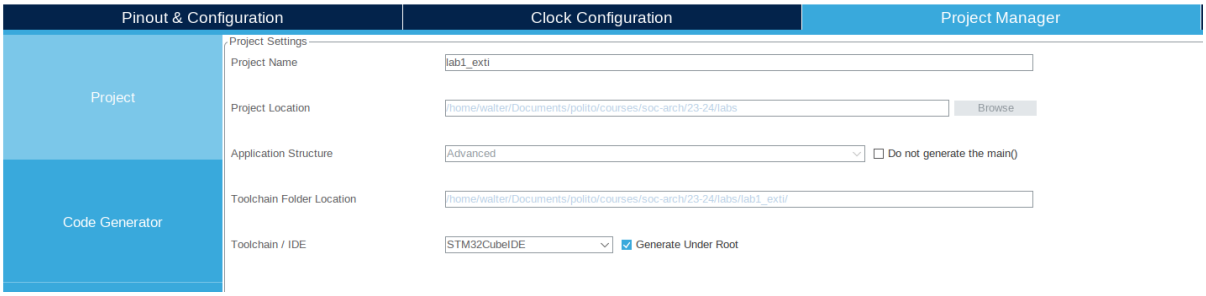


Figure 8: Project Configuration in CubeMX

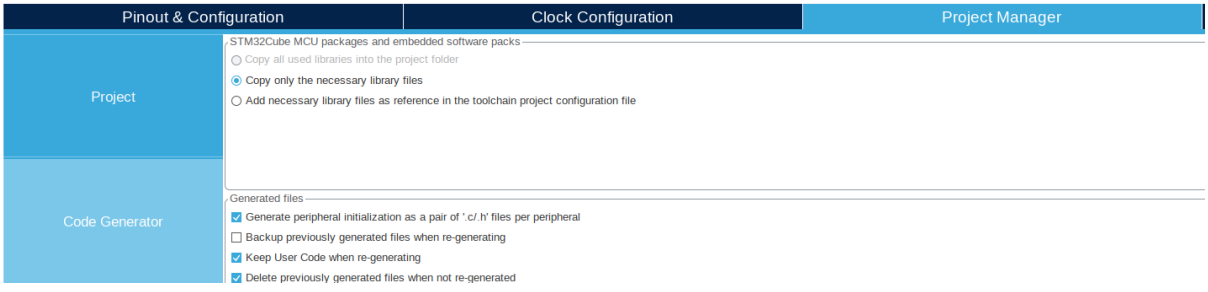


Figure 9: Code Generation in CubeMX

2.2.5 Code generation with stm32pio

Now that the project is configured, we can proceed with the automatic code generation. The structure of the generated code will depend on the IDE we want to use. As you can see from the Toolchain/IDE option, CubeMX natively supports:

- EWARM
- MDK-ARM
- STM32CubeIDE
- Makefile

As we are interested in using renode for running and debugging, it's better to use PlatformIO. (There is no easy support between any of the IDEs above and renode).

To generate the code and project structure for PlatformIO, we will use the stm32pio tool:

1. Open your terminal of choice. Move to the directory where the CubeMX project was saved. Verify that in the directory you have a **.ioc** file.

2. Use **stm32pio** to generate the PlatformIO project. (stm32pio will use CubeMX under the hood).

   First, we verify that stm32pio is able to find the 3 tools that it needs: PlatformIO, STM32CubeMX and Java (for STM32CubeMX) using the **validate** command:

```
stm32pio validate

INFO     checking 'platformio_cmd'...
INFO     checking 'cubemx_cmd'...
INFO     checking 'java_cmd'...
[   ok]  platformio_cmd
[   ok]  cubemx_cmd
[   ok]  java_cmd
```

   Make sure you get OK on all. (On Linux you may get *'java_cmd' is set to None, ignoring*, depending on your configuration. This is ok)

   If you get an error for any of the tools, open the **stm32pio.ini** file that was just generated in the same folder. In the beginning of the file, in the app section, you will find the paths were stm32pio is looking for the tools. For example on Linux :

```
[app]
platformio_cmd = platformio
cubemx_cmd = /home/walter/STM32CubeMX/STM32CubeMX
java_cmd = /home/walter/STM32CubeMX/STM32CubeMX/jre/bin/jar
```

   Note that **platformio_cmd** is set simply to **platformio**. This is why we added the bin folder of platformio to our PATH variable.

   For **cubemx_cmd** and **java_cmd**, you will see a full path. By default, this path points to the **C/Program Files** in windows and the **HOME** folder in Linux. If you installed STM32CubeMX in another path, then you will need to update the stm32pio.ini file with it.

   After correctly updating the file, run again the **validate** command. If the tests pass, you can now generate the code with:

```
stm32pio new -d ./ -b nucleo_f401re
```

   If the result is command not found, replace `stm32pio` with `python -m stm32pio.cli` followed by the same arguments.

If everything works correctly, you should get the following file structure:

```
your_project_name.ioc
Inc
├── gpio.h
├── main.h
├── stm32f4xx_hal_conf.h
├── stm32f4xx_it.h
└── usart.h
lib
└── README
platformio.ini
Src
├── gpio.c
├── main.c
├── stm32f4xx_hal_msp.c
├── stm32f4xx_it.c
└── usart.c
stm32pio.ini
test
└── README
```

NOTE for Windows users: When creating a PlatformIO project for a board that you have never used before, PlatformIO will need to download all the required tools (toolchain) for that specific framework/board. When using stm32pio, this download process will also be triggered. If you have any antivirus or firewall active, it can happen that the download is blocked. You should see some download errors on the terminal. If this happens the resulting PlatformIO project will be incomplete. Open this partial project with VSCode. This should trigger PlatformIO to download the toolchain again, and this time you should also get a pop-up asking for internet permissions for VSCode. Accept and let PlatformIO download what is needed. Then go back to the terminal and launch the **stm32pio new** command again. This time the project should be created correctly.

3. If you need to perform any modifications in CubeMX, you can open your ioc file in the tool and perform the modification. Then, you simply save the project and re-generate the code with stm32pio:

```
stm32pio generate -d ./
```

More details about stm32pio in the README and in the CLI example: https://github.com/ussserrr/stm32pio/tree/master/examples/cli

2.2.6 Personalizing your Firmware

Until now, we configured the hardware part. Now we want to define the behavior, so let's look at the code.

1. Open Visual Studio Code and open your workspace folder. *Trust the Authors*, if requested.
2. Read the code inside **Src_main.c** and focus on the **main()** function. You will find all the initialization functions generated by CubeMX.
3. The firmware is divided into two parts: an initialization part, and an endless loop.
4. If you want to add code, add it between the **USER CODE BEGIN and USER CODE END**. In this way, in case you need to modify the CubeMX configuration (for example, add a peripheral), the tool will preserve your code.
5. What we need to do for our laboratory is to add a callback function that is called as soon as the button is pressed. As explained before, CubeMX is taking care of generating both the IRQ function and the HAL function.
6. Look at the file **stm32f4xx_it.c**. IT stands for Interrupt. You will see the IRQ Handler for the EXTIO interrupt. This function then calls the relative HAL function.

```c
/* USER CODE BEGIN 4 */
/**
* @brief EXTI line detection callbacks.
* @details Toggle the LED each time the callback for the B1_Pin (blue button) is received.
* @param GPIO_Pin Specifies the pins connected EXTI line
* @retval None
*/

void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)

{
  if(GPIO_Pin == B1_Pin)
  {
    HAL_GPIO_TogglePin(LD2_GPIO_Port, LD2_Pin);
  }
}
/* USER CODE END 4 */
```

Listing 2: User EXTI Callback

7. If you follow the function calls (with ctrl + click), you will see the flow is as in the Figure 7:
   (a) `EXTI0_IRQHandler`
   (b) `HAL_GPIO_EXTI_IRQHandler(B1_Pin)`
   (c) `HAL_GPIO_EXTI_Callback(GPIO_Pin)`
8. You will also notice that there is already a Callback function defined in file **Drivers/STM32F4xx_HAL_Driver/Src/stm32f4xx_hal_gpio.c**, as weak:

```c
/**
* @brief EXTI line detection callbacks.
* @param GPIO_Pin Specifies the pins connected EXTI line
* @retval None
*/
__weak void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
/* Prevent unused argument(s) compilation warning */
UNUSED(GPIO_Pin);
/* NOTE: This function Should not be modified, when the callback is needed,
     the HAL_GPIO_EXTI_Callback could be implemented in the user file
*/
}
```

Listing 1: Weak EXTI Callback

9. You need to rewrite the Callback HAL function, to add the behavior we want. As advised in the comment above, we don't modify the callback directly in the Drivers folder. Instead we overwrite it in the main.c file. Because the callback in the Drivers folder is defined as weak, the compiler will prefer the one we define in the main.

   Add the code fragment in listing 2 in the **USER CODE 4** part of the **main.c** file.

   Some notes on the code:

   - B1_Pin, LD4_GPIO_Port, LD4_Pin are labels automatically generated by CubeMX. It is done in the GPIO configuration part. You can personalize them in the *User Label* column, where you can see that the button is labeled as B1 and the LED as LD2.
   - You can see all the labels inside the main.h file.
   - Documentation on the HAL functions, including GPIO Toggle and Callbacks, are available in the HAL documentation for the F4 chip: https://www.st.com/resource/en/user_manual/um1725-description-of-stm32f4-hal-and-lowlayer-drivers-stmicroelectronics.pdf

2.2.7 Compiling

As we saw during the lessons, with PlatformIO we can configure different environments. We want to have two environments, one for running with Renode and one for running with the board.

Remember that it is also possible to create a debug environment, if needed. Students can refer to the corresponding slides.

An example of the **platformio.ini** file follows:

```
[env]
platform = ststm32
board = nucleo_f401re
framework = stm32cube
board_build.stm32cube.custom_config_header = yes

[env:renode]

; Custom upload settings for renode integration
upload_protocol = custom
upload_command = renode $UPLOAD_FLAGS
upload_flags =
    -e include @./nucleo-f401re.resc
    -e sysbus LoadELF @$PROG_PATH
    -e start

[env:board]

[platformio]
include_dir = Inc
src_dir = Src
```

Listing 3: PlatformIO ini file

Note that the first **[env]** is a default one. (no **:name**). This means the variables declared in this environment are inherited by the other envs, like the renode one. This is important as the renode env cannot work without the platform, board and framework variables. Then we can compile and upload either to renode or to the real board. For renode, we use a renode script that in turn will create the machine and load the nucleo platform (required script and platforms for renode are in the provided material).

Before building (or running), remember to choose the correct environment (renode) from the vscode status bar or the PlatformIO side bar. Then hit the build and/or upload button (uploading automatically builds the firmware).

2.2.8 Button and LED in Renode

In Renode you can check the status of the Led, and press / release the user button. Notice how the Status of the Led changes only when the button is pressed and not when it is released.

```
(Nucleo F401RE) sysbus.gpioPortA.UserLed State
False

(Nucleo F401RE) sysbus.gpioPortC.UserButton Press
(Nucleo F401RE) sysbus.gpioPortA.UserLed State
True

(Nucleo F401RE) sysbus.gpioPortC.UserButton Release
(Nucleo F401RE) sysbus.gpioPortA.UserLed State
True

(Nucleo F401RE) sysbus.gpioPortC.UserButton Press
(Nucleo F401RE) sysbus.gpioPortA.UserLed State
False
```

Listing 4: Button and Led in Renode

Finally, you can activate more verbose (NOISY) logs for the Led, with:

```
(Nucleo F401RE) logLevel -1 sysbus.gpioPortA.UserLed
```

You can also add the line above to the platformio.ini so that the logs are enabled automatically.

Then from the terminal renode was launched (in our case the VsCode's *Upload (renode) Task*) you should see the status of the Led each time it changes:

```
17:27:57.6387 [NOISY] gpioPortA.UserLed: LED state changed to True
17:29:17.4129 [NOISY] gpioPortA.UserLed: LED state changed to False
```

2.2.9 (Optional) Testing with the real hardware

Compile and upload to the real hardware and check what happens when you press the user button.

Is the behavior expected?

Hint: Look at the schematic of the Nucleo board https://www.st.com/en/evaluation-tools/nucleo-f411re.html#cad-resources, in particular the user button connection:

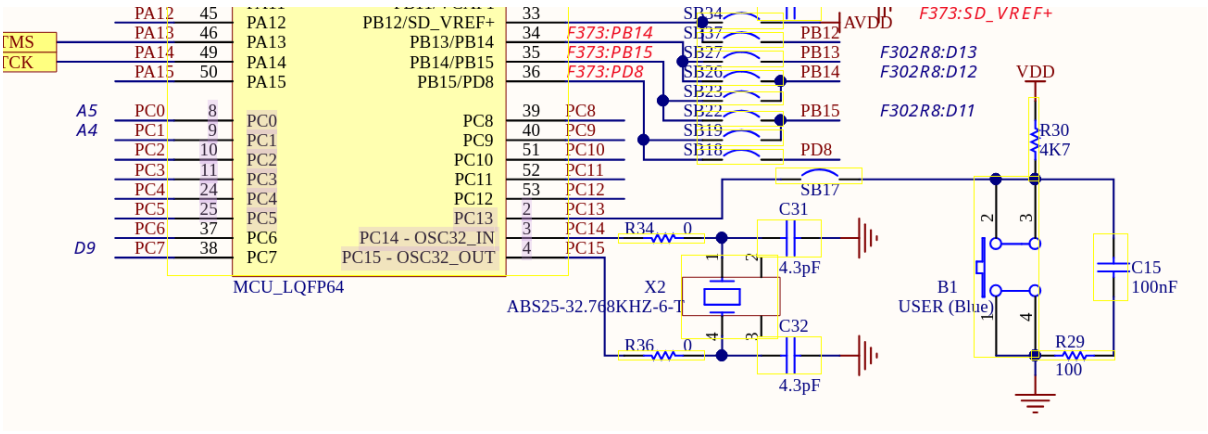


Figure 10: Schematic with focus on user button in the Nucleo F4

3 Homework

3.1 Timers

Timers are powerful MCU peripherals that can keep track of time without using the main processing core. For example, if you want to make a LED blink, you have two possibilities:

- Use the Toggle() function and then the Delay() function inside the main loop. In this way, the processor would be busy for the entire execution period, and the microcontroller will not do anything else.
- Configure a timer that fires at specific intervals. Every time that the timer fires, a callback function turns on/off the LED. Then, the main processing core can go back to what it was doing.

We will use the second option, which is more efficient.
As a reminder, the timer period depends on 3 things:

- The frequency of the time domain where the timer is connected (APBx)
- The value of the prescaler register (PSC)
- The value of the Autoreload register (ARR)

According to the formula:

$$Period = \frac{(ARR + 1) \times (PSC + 1)}{TimerClockFreq} \tag{1}$$

For more information, use the HAL documentation linked before, or other useful documentation you can find online, for example: https://www.waveshare.com/wiki/STM32CubeMX_Tutorial_Series:_Basic_Timer

3.1.1 Fixed blinking frequency

Make the LED blink every 1 second. The suggested steps are:

1. Use CubeMX to configure the GPIO LED.
2. Use CubeMX to configure a timer, define the firing interval, and make it generate interrupts. Use timer10 as the other timers are *advanced timers* which are not yet implemented in renode. You need to find an appropriate value for the ARR and PSC registers, according to the clock frequency of the board.
3. In the firmware, call the Start function for the timer (in the main, before the infinite loop).
4. Write the callback function that will be called whenever the timer fires, and toggle the LED status.

3.1.2 Changing the blinking frequency at runtime

Now that you have a working timer, we want to change the timer frequency at runtime. To do so, we can modify the ARR register (the same one you configured in CubeMX as Counter Period), from the firmware. There are multiple ways of doing this, this is one of them:

```
1 HAL_TIM_Base_Stop_IT(&htim10);
2 __HAL_TIM_SET_AUTORELOAD(&htim10, value);
3 __HAL_TIM_SET_COUNTER(&htim10,0);
4 HAL_TIM_Base_Start_IT(&htim10);
```

Listing 5: Changing frequency using the autoreload register

Note that we are stopping the timer10, setting the new autoreload value, resetting the counter of the timer, and starting the timer. By changing the value, you can change the frequency of the timer.

Requirements:

- The frequency of the timer should change every time the user button is pressed
- At startup, the timer should have a period of 1 second.
- After pressing the button once, the period should change to 2 seconds.
- After pressing it again, the period should change to 10 seconds.
- After pressing it again, the period should go back to 1 second, and the sequence repeats
- You are not allowed to use polling or delay strategies, instead you should use the interrupts paradigm.

Hints:

- Configure the button to generate an interrupt, Then in the interrupt, change the frequency of the timer.
- The timers are not started automatically by the firmware. If the LED status is not changing, maybe you forgot to start the timer in the main.
- You can check the status of the timer from Renode.
- You will probably need to use a global variable to keep track of the ARR value to set.

A note on Renode simulation and timers: Given how the Renode simulation works, the period you configure in CubeMX or in the firmware will not correspond to the period you see on the simulation (the frequency of the logs you will see on the terminal). Don't pay too much attention to this, what is important is that you see the difference in the logs frequency when you change timer frequency. Additionally, the simulation is typically slower than the real board execution (this will also depend on how powerful your PC is). If the Led status is not changing, maybe it is just renode that is taking too much time to simulate. Try using a smaller period (divide by 10 or 100) and try again.

3.2 DMA and UART

3.2.1 DMA self-study

You will learn the STM32 DMA peripherals basics by looking at an example from ST and reproducing it in your environment.

1. Download the STM32Cube package for the STM32F4-based products. You will download a zipped folder with a lot of examples and documentation about them. Feel free to wander! https://www.st.com/en/embedded-software/stm32cubef4.html
2. Identify the DMA - FlashToRam example for the F4 discovery board (There is no equivalent example for the nucleo board, but the concepts are the same).

   Projects/STM32F4-Discovery/Examples/DMA/DMA_FLASHToRAM/
3. Open it in Visual Studio Code. Focus on the **Src** and **Inc** folders.
4. This firmware example shows how to configure the Direct Memory Access peripheral to transfer data from the flash memory of the MCU to the RAM **without keeping the processor busy**: in this way, the computational unit can continue working on something else while the data is transferred. This is fundamental because memories are slower than processing units.
5. Check the **main.c** file. Be sure to take a good look at the following:

   (a) **aSRC_Const_Buffer** is saved in the flash, while **aDST_Buffer** is free space allocated in the RAM, ready to accept data from the DMA. Check how they are declared differently so that the compiler assigns them to the correct memory space.

   (b) The **DMA_Config()** function contains all the properties needed for the configuration of the DMA.

(c) The DMA is configured in interrupt mode: once it finishes, it alerts the CPU via interrupt. Of course, you need to define functions that will be called as soon as the interrupt is triggered. In this case, they defined **TransferComplete** and **TransferError**, and they are linked to the DMA configuration via **HAL_DMA_RegisterCallback** function.

(d) The transfer is then started when the configuration is completed via the **HAL_DMA_Start_IT** function.

3.2.2 Adding the UART

Requirements

1. Create a new CubeMX project.
2. Configure the DMA2 component with the properties you found in the **DMA_Config()** function.
3. Configure the NVIC enabling the DMA2 Interrupts.
4. Configure the USART2 peripheral in Asynchronous mode.
5. Save the CubeMX project.
6. Autogenerate the code for a pio project using stm32pio.
7. Personalize the firmware by using what you learnt:
   (a) Define a personalized message in the FLASH. You can use whatever content content and the size you prefer.
   (b) Allocate the corresponding space in SRAM.
   (c) Create prototypes for your Transfer Complete/Error callback functions.
   (d) Use the **HAL_DMA_RegisterCallback()** to register your callbacks
   (e) Use the **HAL_DMA_Start_IT()** function to start the transfer to the allocated SRAM space.
   (f) Inside your Transfer Complete callback function, add the lines of code needed to transmit the message on UART (reading it from the SRAM). You can find them in the STM32F4 HAL documentation.
   (g) Inside your Transfer Error callback function, turn ON an LED to signal the error.
8. Simulate it via Renode. As we saw during the Renode lessons, you can create a UART window to communicate with your firmware by adding the following line to your Renode script:

   showAnalyzer usart2

Hints

- Be careful to where you call **The HAL_DMA_Start_IT()**:
  - If you put it **before** the **configuration of the UART** device, the DMA transfer may take less time than the UART configuration, and at the end, you won't see anything on the UART. This is a common mistake, and it could even work on Renode but not on the real hardware (as the timing is different between the hardware and the simulator).
  - It is a good idea to launch it explicitly after all the configuration functions. In this way, the code is clearer to read and understand.
  - In the self-study example, it is placed at the end of the DMA configuration part. This is a design choice, and it works because there is no dependency with other peripherals, but in our case, we have a dependency on the UART peripheral.
- When testing with the real hardware, notice that in the Nucleo Board, the UART2 is exposed through the st-link usb connection, so there is no need to connect anything to the pins of the board, just the USB cable is enough.

  There are multiple ways to access the UART from your computer. You can easily access it using the PlatformIO **serial Monitor**. If that does not work, you can try:

  Windows:

- Use a third party program like the popular **Putty**
- Search for the Device Manager and open it. Look the current devices under the Ports (**COM** and LPT) section.
- Connect the board. A new **COMxx** device will appear.
- Open Putty. Perform a Serial connection to the **COMxx** device you connected. Speed and the other properties should be configured in the same way you configured the UART in CubeMX.

Linux:

- Open a terminal and launch **ls /dev/ — grep tty**
- Connect the board. Launch the command again, and note the new device that appears. Usually it is something like **ttyACMx** or **ttyUSBx**.
- Launch **screen /dev/ttyXXXx baudrate** , putting in the arguments the correct device you found and the baudrate you configured in CubeMX
- (An alternative to screen is minicom)
- To exit **screen**, press Ctrl+A and then Ctrl+/

3.3 Interrupt priorities

In this last exercise, you will understand why interrupt priorities are important and how to manage them.

1. Add a call to **HAL_Delay(100)** inside the Transfer Complete callback, just before UART communication. This should delay the writing on the UART interface by less than a second.
2. Is it working, or does it seem to be waiting forever?
3. If it is waiting forever, this is happening because we configured the DMA Interrupt and the SysTick Interrupt (used by the Delay function) with the same preemption priority.
   - The DMA transfer complete callback is waiting for the Delay function to finish.
   - At the end of the Delay function, a SysTick interrupt is generated.
   - The DMA interrupt should be **preempted** for the CPU to handle the SysTick Interrupt.
   - But the SysTick Interrupt **is not handled, because it has the same preemption priority** as the DMA interrupt that is currently in execution.
   - But the Delay function will be terminated ONLY once the SysTick interrupt is received
   - We are in an interrupt **deadlock**.

Requirement Using CubeMX (or directly the firmware if you feel brave), solve the deadlock issue.

Hints:

- it's probably not a good idea to mess with the SysTick interrupt.
- If you have used CubeMX to update some peripheral configuration, remember to update the code with stm32pio and the generate option.

4 Troubleshooting

These are some common issues that we have seen during the development of this lab. If you have any other issues, please let us know so we can add them to the list.

4.1 stm32pio issues

stm32pio cannot find CubeMX. stm32pio looks for the CubeMX installation in the default folders. For Linux, under **/home/STM32CubeMX**. For Windows, under **Program Files**. Verify your installation of CubeMX. If it is not in the default folder, you can either tell stm32pio where it is, with a configuration file (check the stm32pio documentation), or simply uninstall and install CubeMX in the correct (default) folder (for all users for windows)

stm32pio cannot find platformio. smt32pio expects the **pio** command to be accessible from a terminal. That is why we added it to the Path env variable. Open a new terminal and verify that you can execute the pio command. If not, there is something wrong with your Path env variable. Follow the steps described in section 2.1.2.

Command not found. If the result of running stm32pio is *command not found*, instead of calling stm32pio directly, try calling it with python, as **python -m stm32pio.cli**. If this does not work either, make sure stm32pio is actually installed in the currently active Python environment, with **pip list**, and that the version of stm32pio is the latest one.

4.2 PlatformIO issues

Missing variables in environments. Some variables are required for all environments (platform, board, framework). Make sure your envs have them. In the example ini file in this pdf, we use a default env with no name, so that the variables are inherited by all other envs.

Are you compiling and uploading the correct environment? Remember that in platformio you can have multiple environments, and you must choose the appropriate one before compiling and uploading, using the button in the status bar (or the menu options). See section 2.2.7 for more details.

4.3 Renode simulation issues

Spaces. If the renode simulation does not work, for example when you press/release the button, the LED does not turn on, it could be that there are spaces in the path where you saved your project. Try copying your project to another location without spaces (for example /home/ or the C:) and run it again from there.

Renode does not start If platformio is not able to find renode, it will not be able to start the simulation. Verify that you can actually start renode from a terminal, by simply typing **renode**.

- If it does not work from a terminal, there is a problem with your path env variable, fix it to include the renode installation folder.
- If it works from a terminal, perhaps your vscode instance has an old version of the path variable. Close vscode and open it again. (If possible from the same terminal from which you were able to execute renode).